

CQRS Pattern in System Design

 systemdesignschool.io/fundamentals/cqrs



Command Query Responsibility Segregation (CQRS) Pattern

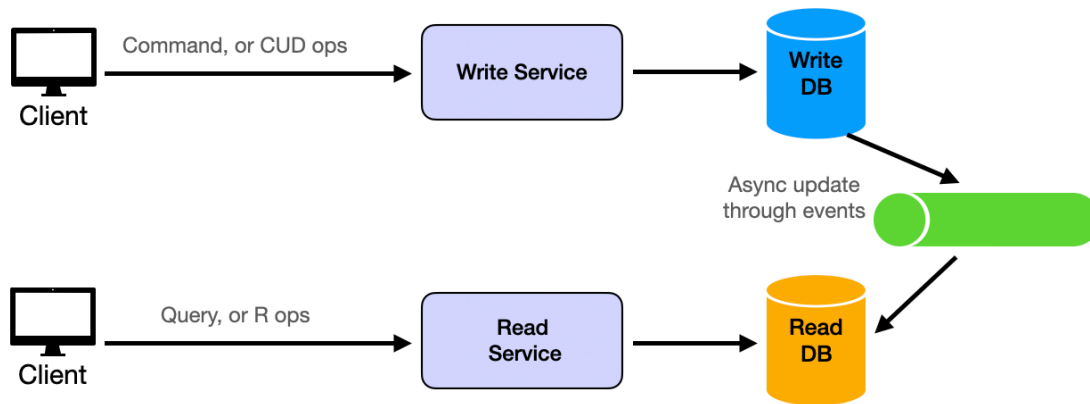
Introduction and Motivation

Since read and write typically have different traffic patterns, we can optimize the system by handling read and write operations in different ways, and scale them independently. The idea is similar to database replication except CQRS does it on the application level.

How the CQRS Pattern Works

Separating Commands and Queries

CQRS draws a clear line between actions that change the state of the system—known as **commands (mutations or CUD)**—and operations that retrieve data—known as **queries (reads)**.



CRUD = Create, Read, Update, Delete



Eventual Consistency

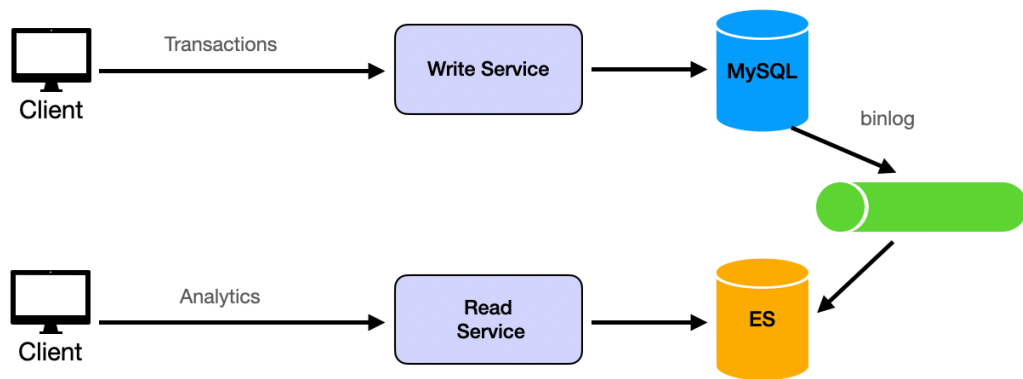
A key component of many CQRS implementations is eventual consistency. When a command is executed, the system emits events that describe the change that just occurred. These **events update read-optimized models asynchronously**, ensuring that the write side is never slowed down by the need to immediately reflect these changes in query results. While this can lead to transient differences between the write and read representations, it allows each side to evolve independently and efficiently.

Real-world Examples

MySQL and Elasticsearch Implementation

A classic combination for systems requiring both ACID compliance and full-text search:

- **Command Side:** MySQL handles transactional data:
 - Order processing and inventory management
 - User account operations
 - Financial transactions
 - Referential integrity enforcement
- **Query Side:** Elasticsearch powers search functionality:
 - Full-text product search
 - Complex filter combinations
 - Faceted navigation
 - Real-time search suggestions
 - Analytics and reporting



DynamoDB with Different Access Patterns

The command and query side can also be the same database, storing different tables. The **query side is essentially a denormalized view** of the data optimized for reads.

A notification system using CQRS provides several key advantages:

Command Side:

Simple, fast writes for new notifications

```
// notifications-write table - optimized for writes { "notification_id": "uuid-123", "user_id": "user_789", "type": "friend_request", "sender_id": "user_456", "created_at": "2024-03-20T10:30:00Z", "status": "unread" }
```

- Fast inserts without complex data transformations
- No need to update aggregates on write

Query Side:

Pre-computed views for efficient reads

```
// notifications-read table - optimized for reads { "user_id": "user_789#2024-03", // Partition key for monthly feeds "unread_count": 5, "notifications": [ { "id": "uuid-123", "type": "friend_request", "sender": { "id": "user_456", "name": "John Doe", "avatar_url": "https://..." }, "created_at": "2024-03-20T10:30:00Z", "status": "unread" } // ... more notifications ] }
```

- Pre-joined data eliminates need for multiple queries
- Pre-calculated aggregates (unread_count)
- Optimized for specific access patterns
- Can scale reads independently

Without CQRS, you might face these challenges:

1. Complex writes that need to update multiple aggregates
2. Slow reads requiring joins across multiple records
3. Table design compromises trying to serve both write and read patterns
4. Scaling issues when read and write loads differ significantly

With CQRS:

1. Write service focuses on recording notifications quickly
2. Async processor enriches data (e.g., fetching sender details)
3. Read model maintains pre-computed views
4. Each side can scale independently based on its specific load

Pros and Cons of CQRS

Advantages

- **Faster Reads:** The read model can be optimized specifically for queries by using denormalized data structures and pre-computed views. Since read and write operations are separated, each can be scaled independently based on their specific load patterns. This means heavy read traffic won't impact write performance and vice versa.
- **Enables Complex Queries:** CQRS allows using specialized databases for different purposes - for example, using Elasticsearch for full-text search while keeping transactional data in MySQL. This flexibility enables complex querying capabilities without compromising write performance.
- **Decouples Read/Write Operations:** Read and write models can evolve independently based on their specific requirements. Teams can choose the most appropriate technology for each side and optimize them separately, leading to better overall system performance.

Drawbacks

- **Complexity:** Implementing CQRS adds significant complexity to the system. Teams need to maintain multiple models and handle synchronization between them. This complexity can make the system harder to understand and maintain.
- **Eventual Consistency:** Since updates to the read model happen asynchronously, there's always a period where the read side may show outdated data. Applications need to be designed with this in mind, and users need to understand that changes might not be immediately reflected.
- **Infrastructure Overhead:** Running multiple databases and message queues requires additional infrastructure and operational costs. Teams need to manage more components and handle the complexity of coordinating deployments across these different systems.

Lightweight CQRS Variants

Not every CQRS deployment needs two databases and an asynchronous projection pipeline. In smaller systems the command and query sides often share the same datastore and keep their models in sync within a single transaction. This preserves strong consistency and reduces infrastructure overhead while still letting you maintain separate command/query models in the application layer. Teams frequently evolve from this lightweight form to the more common "commands in the OLTP store, queries in a read store" architecture only when scaling pressures or specialized query workloads justify the extra moving parts.

Implementing CQRS on Popular Cloud Platforms

On Azure, you might implement the write side with Azure Functions or Azure App Services to process commands, while publishing domain events to Azure Service Bus or Event Grid. The read side could use Azure Cosmos DB or Azure SQL Database for storing denormalized views, with another set of Functions handling event projections and maintaining query-optimized data structures.

Amazon Web Services (AWS)

With AWS, you can utilize AWS Lambda functions to handle commands and emit events via Amazon SNS or SQS. The read side might use Amazon DynamoDB, enhanced by Global Secondary Indexes, for fast queries. You can also incorporate Amazon ElastiCache or Amazon Elasticsearch Service to speed up queries, while AWS EventBridge helps coordinate event flows and updates.

Google Cloud Platform (GCP)

In GCP, Google Cloud Functions can process commands and publish events through Pub/Sub. These events trigger additional Cloud Functions to update read-optimized structures in Cloud Firestore or BigQuery. This combination allows you to achieve high query performance for analytics and reporting while keeping domain logic neatly separated.